

Framework Topology Statistics Specification

Normative TS2+TS3 API for Framework Descriptors, Bridge Signatures, Exact Timelines, and Projected-Edge Episodes in mdstats

mdstats

2026-07-14 (TS3 integration revision)

Contents

1	Purpose and status	2
2	Motivation	3
3	Responsibility boundaries	3
3.1	Owned by <code>framework.py</code>	3
3.2	Not owned by <code>framework.py</code>	3
4	Scientific definitions	4
4.1	Topology-class occupancy	4
4.2	Basic graph descriptors	4
4.3	Endpoint-species counts	4
4.4	Whole-path bridge signatures	5
4.5	Projected-edge occupancy	5
4.6	Degree statistics	5
5	Public constants	5
6	Public data structures	6
6.1	<code>FrameworkStatisticsOptions</code>	6
6.1.1	TS3 temporal options	6
6.2	<code>FrameworkBridgeSignature</code>	6
6.3	<code>FrameworkGraphDescriptorStatistics</code>	6
6.4	<code>FrameworkEndpointPairStatistics</code>	7
6.5	<code>FrameworkBridgeSignatureStatistics</code>	7
6.6	<code>FrameworkEdgeOccupancy</code>	7
6.7	<code>FrameworkVertexDegreeStatistics</code>	7
6.8	Aggregate transition structures	8
6.9	<code>FrameworkTemporalStatistics</code>	8
6.10	<code>FrameworkTopologyStatistics</code>	8
7	Public function	9
7.1	Input contract	9
7.2	Output contract	10
8	Algorithm	10
8.1	State-compressed descriptor evaluation	10
8.2	Complexity	10

9	Trajectory and ensemble semantics	11
9.1	Trajectory	11
9.2	Ensemble	11
9.3	Per-frame catalog mode	11
10	Serialization and provenance	11
11	Edge cases and warnings	11
11.1	Cycle rank is not ring count	11
11.2	Asymmetric linker order	11
11.3	Parallel edges	11
11.4	Self-image edges	11
11.5	Missing transition differences	12
11.6	Constant series	12
11.7	Correlated frames	12
11.8	Large edge unions	12
12	Validation requirements	12
13	Na-LTA acceptance case	12
14	Deferred features	13
15	Implementation checklist	13
16	Summary	13

1 Purpose and status

This document specifies the implemented module

`mdstats/analysis/topology_statistics/framework.py`

and its public re-export surface through

`mdstats.analysis.topology_statistics`

`mdstats.analysis`

`mdstats`

for topology-statistics Stage TS2. The implementation is introduced in `mdstats 0.17.0a2`; exact TS3 temporal integration is added in `0.17.0a3`. TS4 consumes the completed framework result unchanged in `0.17.0a4` for exact cross-layer alignment.

TS2 consumes one completed `TopologyCatalog` and derives descriptive framework- graph statistics. It does not rebuild atomic connectivity, reproject framework paths, reconcile topology identity, or enumerate primitive rings.

The governing boundary is

the topology catalog defines framework identity; TS2 only summarizes it .

The initial implementation provides:

- exact topology-class occupancy and Shannon diversity;
- per-frame framework vertex, edge, component, isolated-vertex, self-image-edge, parallel-edge, and cycle-rank descriptors;
- exact endpoint-species edge-count distributions;
- whole-path orientation-aware bridge-signature distributions;
- species-resolved framework degree statistics;
- canonical projected-edge occupancy;

- trajectory-only aggregate framework additions, removals, and affected atoms;
- immutable, schema-versioned result objects;
- deterministic SHA-256 result digests;
- optional exact TS3 class timelines and projected-edge episodes for trajectories.

Exact class timelines, residence intervals, transition matrices, return lags, and canonical projected-edge episodes are supplied by the integrated TS3 layer.

2 Motivation

A `TopologyCatalog` stores exact framework classes and frame assignments, but it is not itself a compact statistical report. TS2 converts that authoritative catalog into scalar distributions and frame-aligned series suitable for validation, comparison, plotting, and later cross-layer analysis.

Typical questions include:

- Is the framework topology uniform across all frames?
- How many vertices, projected edges, and connected components occur?
- Are all framework vertices four-coordinate?
- Which endpoint-species pairs and linker-path chemistries occur?
- Are projected edges permanent or intermittent?
- Does a topology change split components or alter the cycle-space rank?
- Which framework vertices and linker atoms participate in transitions?

For the validated 300 K Na-LTA trajectory, TS2 reports one framework class over 2,000 frames, 48 T vertices, 96 projected T–T edges, one component, degree four for every Si and Al vertex, 96 Al–Si bridges with Al–O–Si whole-path signature, and occupancy one for all 96 projected edges.

3 Responsibility boundaries

3.1 Owned by `framework.py`

The module owns:

- topology-class occupancy and diversity through TS0 primitives;
- exact per-topology and per-frame graph descriptors;
- endpoint-species projected-edge counts;
- complete whole-path bridge-signature counts;
- framework-vertex degree statistics by species and atom;
- canonical `FrameworkEdgeKey` occupancy;
- aggregate trajectory transition effects when edge differences are stored;
- source catalog, mapping, and topology provenance;
- TS2 serialization and digest validation.

3.2 Not owned by `framework.py`

The module must not:

- calculate geometric neighbors;
- define atomic connectivity;
- assign framework roles or path rules;
- project atomic paths into framework edges;
- merge or split topology classes;
- smooth transient topology assignments;
- enumerate primitive, strong, or cage rings;
- interpret graph cycle rank as a primitive-ring count;
- infer physical time when no time array is supplied;
- treat ensemble storage order as time;

- infer independent-sample uncertainties from correlated trajectory frames.

4 Scientific definitions

4.1 Topology-class occupancy

For topology class k with frame count n_k in F analyzed frames,

$$p_k = \frac{n_k}{F}.$$

The Shannon diversity is

$$H = - \sum_k p_k \ln p_k,$$

and the effective topology count is

$$N_{\text{eff}} = \exp(H).$$

For a uniform framework, $H = 0$ and $N_{\text{eff}} = 1$.

4.2 Basic graph descriptors

For topology $G = (V, E)$ with C connected components, TS2 reports:

- vertex count $|V|$;
- projected multiedge count $|E|$;
- connected-component count C ;
- isolated-vertex count;
- projected self-image-edge count;
- number of endpoint pairs with multiplicity greater than one;
- excess parallel-edge count $\sum_{uv} \max(0, m_{uv} - 1)$;
- graph cycle-space rank

$$\beta_1 = |E| - |V| + C.$$

The quantity β_1 is the dimension of the graph cycle space. It is not the number of primitive rings and must never be labeled as such.

4.3 Endpoint-species counts

For unordered framework endpoint species A and B ,

$$N_{AB}(f) = |\{e \in E_f : \{Z_{e,i}, Z_{e,j}\} = \{A, B\}\}|.$$

These counts do not encode internal linker chemistry.

4.4 Whole-path bridge signatures

A projected edge path has the chemical sequence

$$\Sigma = (Z_i, Z_{\ell_1}, \dots, Z_{\ell_m}, Z_j).$$

The signature is canonical only under reversal of the complete path:

$$[\Sigma] = \min_{\text{lex}}(\Sigma, \Sigma^{-1}).$$

Thus,

$$A - \text{O} - \text{S} - B \equiv B - \text{S} - \text{O} - A,$$

but

$$A - \text{O} - \text{S} - B \neq A - \text{S} - \text{O} - B.$$

TS2 includes `rule_id` and `edge_kind` in `FrameworkBridgeSignature`; two chemically identical species sequences declared by scientifically different path rules remain distinguishable.

4.5 Projected-edge occupancy

For canonical `FrameworkEdgeKey` e ,

$$p_e^{\text{F}} = \frac{1}{F} \sum_{f=1}^F \mathbf{1}[e \in E_f].$$

Stage 2 has already normalized periodic gauge and complete path orientation. TS2 therefore uses the canonical edge key directly; it must not replace the key with only an unordered endpoint pair.

4.6 Degree statistics

For framework vertex i in frame f , let $d_i(f)$ be the multigraph degree stored by `FrameworkTopology`. For species A , the pooled exact degree PMF is

$$P_A(k) = \frac{1}{FN_A} \sum_f \sum_{i \in A} \mathbf{1}[d_i(f) = k].$$

TS2 also reports the mean degree per frame and the population mean and standard deviation for every persistent vertex.

5 Public constants

```
CANONICAL_FRAMEWORK_TOPOLOGY_STATISTICS_SCHEMA = (  
    "mdstats.topology-statistics.framework.v2"  
)
```

TS2 uses the shared digest algorithm

```
TOPOLOGY_STATISTICS_DIGEST_ALGORITHM = "sha256"
```

and supports only source schemas compatible with the current `TopologyCatalog` and `FrameworkTopology` implementations.

6 Public data structures

6.1 FrameworkStatisticsOptions

```
@dataclass(frozen=True, slots=True)
class FrameworkStatisticsOptions:
    include_degree_statistics: bool = True
    include_edge_occupancies: bool = True
    include_transition_statistics: bool = True
    quantiles: tuple[float, ...] = DEFAULT_QUANTILES
```

Constraints:

- all inclusion flags are boolean;
- quantiles are finite, strictly increasing, start at 0.0, and end at 1.0;
- disabling an optional result prevents its potentially large records from being materialized.

6.1.1 TS3 temporal options

FrameworkStatisticsOptions additionally contains:

```
include_temporal_statistics: bool = True
include_edge_episodes: bool = True
temporal_options: TemporalStatisticsOptions = TemporalStatisticsOptions()
```

Detailed temporal output is created only for trajectory input. Edge episodes may be disabled independently when the union of projected framework edges is large.

6.2 FrameworkBridgeSignature

```
@dataclass(frozen=True, order=True, slots=True)
class FrameworkBridgeSignature:
    path_atomic_numbers: tuple[int, ...]
    rule_id: str
    edge_kind: str
```

Constraints:

- the path contains at least two valid atomic numbers;
- the complete path is canonicalized against its complete reverse;
- rule_id and edge_kind are nonempty;
- linker order is never independently sorted.

Convenience constructor:

```
FrameworkBridgeSignature.from_symbols(
    path_symbols,
    *,
    rule_id,
    edge_kind="framework",
)
```

6.3 FrameworkGraphDescriptorStatistics

```
@dataclass(frozen=True, slots=True)
class FrameworkGraphDescriptorStatistics:
    descriptor: str
    topology_values: NDArray[np.int64]
    series: ScalarSeries
    distribution: DiscreteCountDistribution
```

Supported descriptor names are:

```
vertex_count
edge_count
component_count
isolated_vertex_count
self_image_edge_count
parallel_endpoint_pair_count
parallel_edge_excess_count
cycle_rank
```

`topology_values[k]` is evaluated once for topology class *k*. `series` is the frame-level expansion through `frame_topology_ids`.

6.4 FrameworkEndpointPairStatistics

```
@dataclass(frozen=True, slots=True)
class FrameworkEndpointPairStatistics:
    species_pair: tuple[int, int]
    topology_edge_counts: NDArray[np.int64]
    edge_count_series: ScalarSeries
    edge_count_distribution: DiscreteCountDistribution
```

The pair is canonical and unordered. It summarizes only projected-edge endpoint species.

6.5 FrameworkBridgeSignatureStatistics

```
@dataclass(frozen=True, slots=True)
class FrameworkBridgeSignatureStatistics:
    signature: FrameworkBridgeSignature
    topology_edge_counts: NDArray[np.int64]
    edge_count_series: ScalarSeries
    edge_count_distribution: DiscreteCountDistribution
```

This object preserves endpoint-linker orientation coupling.

6.6 FrameworkEdgeOccupancy

```
@dataclass(frozen=True, slots=True)
class FrameworkEdgeOccupancy:
    edge_key: FrameworkEdgeKey
    frame_count: int
    probability: float
```

Constraints:

- `edge_key` is the exact Stage 2 canonical decorated key;
- $0 \leq \text{frame_count} \leq F$;
- $\text{probability} = \text{frame_count}/F$.

6.7 FrameworkVertexDegreeStatistics

```
@dataclass(frozen=True, slots=True)
class FrameworkVertexDegreeStatistics:
    atomic_number: int
    vertex_atom_indices: NDArray[np.int64]
    degree_distribution: DiscreteCountDistribution
    mean_degree_series: ScalarSeries
```

```

per_vertex_mean_degree: NDArray[np.float64]
per_vertex_population_standard_deviation: NDArray[np.float64]

```

Atom indices are persistent canonical collection indices.

6.8 Aggregate transition structures

```

@dataclass(frozen=True, slots=True)
class FrameworkEndpointPairTransitionCount:
    species_pair: tuple[int, int]
    additions: int
    removals: int

@dataclass(frozen=True, slots=True)
class FrameworkBridgeTransitionCount:
    signature: FrameworkBridgeSignature
    additions: int
    removals: int

@dataclass(frozen=True, slots=True)
class FrameworkTransitionAggregateStatistics:
    n_frame_boundaries: int
    n_changed_boundaries: int
    total_added_edges: int
    total_removed_edges: int
    endpoint_pair_counts: tuple[FrameworkEndpointPairTransitionCount, ...]
    bridge_signature_counts: tuple[FrameworkBridgeTransitionCount, ...]
    affected_vertex_atom_indices: NDArray[np.int64]
    affected_vertex_event_counts: NDArray[np.int64]
    affected_linker_atom_indices: NDArray[np.int64]
    affected_linker_event_counts: NDArray[np.int64]

```

This remains a trajectory-wide aggregate; the integrated TS3 result owns boundary-resolved temporal records.

6.9 FrameworkTemporalStatistics

```

@dataclass(frozen=True, slots=True)
class FrameworkTemporalStatistics:
    state_statistics: StateTransitionStatistics
    edge_keys: tuple[FrameworkEdgeKey, ...]
    edge_episodes: EntityPresenceStatistics | None

```

`edge_keys[e]` gives the canonical Stage 2 projected edge represented by dense entity ID `e`. Identity includes the complete decorated path and preserves whole-path reversal equivalence.

The convenience method

```
edge_episode_statistics(edge_key: FrameworkEdgeKey)
```

returns all episodes for one projected framework edge.

6.10 FrameworkTopologyStatistics

```

@dataclass(frozen=True, slots=True)
class FrameworkTopologyStatistics:
    axis: FrameAxis
    catalog_occupancy: CatalogOccupancyStatistics
    vertex_atom_indices: NDArray[np.int64]
    vertex_atomic_numbers: NDArray[np.int64]
    graph_descriptors: tuple[FrameworkGraphDescriptorStatistics, ...]

```



```

endpoint_pair_statistics: tuple[FrameworkEndpointPairStatistics, ...]
bridge_signature_statistics: tuple[FrameworkBridgeSignatureStatistics, ...]
degree_statistics: tuple[FrameworkVertexDegreeStatistics, ...] | None
edge_occupancies: tuple[FrameworkEdgeOccupancy, ...] | None
edge_occupancy_summary: ScalarSummary | None
transition_statistics: FrameworkTransitionAggregateStatistics | None
temporal_statistics: FrameworkTemporalStatistics | None
options: FrameworkStatisticsOptions
source_catalog_schema: str
source_framework_schema: str
source_catalog_digest: str
source_mapping_digest: str
source_topology_digests: tuple[str, ...]
metadata: Mapping[str, object]
canonical_schema_version: str
digest_algorithm: str
digest: str

```

Convenience queries:

```

result.descriptor("edge_count")
result.endpoint_pair("Si", "Al")
result.bridge_signature(signature)
result.species_degree("Si")

```

All identity-bearing arrays are defensive read-only copies.

7 Public function

```

def compute_framework_topology_statistics(
    catalog: TopologyCatalog,
    *,
    steps: ArrayLike | None = None,
    times: ArrayLike | None = None,
    time_unit: str | None = None,
    options: FrameworkStatisticsOptions | None = None,
) -> FrameworkTopologyStatistics:
    ...

```

7.1 Input contract

catalog must be a completed and internally valid TopologyCatalog.

All stored topology classes must:

- use the supported framework-topology schema;
- share the same persistent framework vertex atom indices;
- share the same vertex atomic numbers by index;
- use the catalog mapping digest;
- preserve canonical FrameworkEdgeKey ordering.

This shared-vertex condition follows the current fixed FrameworkMapping and atom-identity contract. Catalogs spanning incompatible atom populations must be split or explicitly remapped before TS2.

steps and times are optional display axes. They must align with the selected catalog frames. Physical time is never inferred from frame IDs. Ensemble results reject temporal axes.

7.2 Output contract

The output contains descriptors evaluated once per unique topology and expanded exactly through the catalog assignment. No graph is reconstructed or modified.

8 Algorithm

8.1 State-compressed descriptor evaluation

Let K be the number of topology classes and F the number of frames.

1. Validate the catalog and common vertex identity.
2. Build the frame/sample axis.
3. Compute class occupancy using `frame_topology_ids`.
4. For each topology class, evaluate all graph descriptors once.
5. Enumerate endpoint pairs and bridge signatures across the class union.
6. Count each category once per topology class.
7. Expand class-level scalar values to frames.
8. Accumulate degree statistics using topology frame counts.
9. Accumulate exact edge occupancy using topology frame counts.
10. For trajectories with stored transitions, aggregate additions, removals, and affected vertices/linkers.
11. Freeze arrays, serialize provenance, and calculate the SHA-256 digest.

Pseudocode:

```
validate catalog and topology alignment
axis <- build_frame_axis(...)
occupancy <- occupancy(frame_topology_ids)

for topology in unique_topologies:
    descriptors[topology] <- graph_descriptors(topology)
    endpoint_counts[topology] <- count_endpoint_pairs(topology)
    bridge_counts[topology] <- count_whole_path_signatures(topology)
    degrees[topology] <- topology.degree

frame_series <- expand(class_values, frame_topology_ids)
edge_occupancy <- weighted union of canonical edge keys
transition_aggregate <- optional trajectory reduction
return immutable FrameworkTopologyStatistics
```

8.2 Complexity

Let E_k and V_k be the edge and vertex counts of topology class k . Descriptor extraction scales as

$$O\left(\sum_{k=1}^K (V_k + E_k)\right).$$

Frame expansion scales as

$$O(FD),$$

where D is the number of requested scalar series.

Edge occupancy scales with the union of canonical projected edges, not with $F \max_k E_k$. This is the principal benefit of consuming the catalog rather than reanalyzing every frame graph.

9 Trajectory and ensemble semantics

9.1 Trajectory

TS2 may report aggregate transition effects because `TopologyCatalog.transitions` has physical adjacent-frame meaning. It does not yet expose exact event times, dwell intervals, or transition matrices.

9.2 Ensemble

TS2 reports class occupancy, graph-descriptor distributions, degree statistics, and edge occupancy. It must not compute transitions, visits, lifetimes, or correlations from stored sample order.

9.3 Per-frame catalog mode

A trajectory catalog built with `mode="per_frame"` may be summarized statically. Because Stage 3 deliberately stores no reconciled transition records in this mode, TS2 returns `transition_statistics=None`.

10 Serialization and provenance

`FrameworkTopologyStatistics.to_dict()` stores:

- TS2 schema and digest algorithm;
- frame axis and class occupancy;
- all enabled descriptor results;
- source catalog, mapping, and topology digests;
- source schema identifiers;
- options and metadata;
- the final result digest.

`FrameworkTopologyStatistics.from_dict()` validates the schema, reconstructs immutable result objects, and recomputes the digest. A modified payload is rejected.

The digest is a reproducibility identifier for the statistical result. It does not replace structured equality of the source framework topology.

11 Edge cases and warnings

11.1 Cycle rank is not ring count

A connected graph with $E - V + 1 = 49$ has cycle-space dimension 49. It does not necessarily contain 49 primitive rings under the removed-edge shortest-path definition. Ring enumeration remains Stage S4 of the framework/ring track.

11.2 Asymmetric linker order

Bridge statistics must never independently sort endpoint species and linker species. `A-O-S-B` and `A-S-O-B` are distinct unless separately declared.

11.3 Parallel edges

Parallel projected edges are valid because distinct linker paths or periodic instances may connect the same canonical vertices. TS2 reports both the number of endpoint pairs with multiplicity and the number of excess edge instances.

11.4 Self-image edges

A projected self-image edge has identical canonical endpoint atom indices and a nonzero periodic translation. It is not an invalid zero-length self-loop. TS2 counts it separately while retaining its exact edge key.

11.5 Missing transition differences

If Stage 3 was configured with framework-edge difference storage disabled, TS2 cannot reconstruct edge additions or removals from an empty transition payload. Affected vertex and linker sets remain available, but edge-category totals may be zero. The result metadata and source catalog options remain authoritative.

11.6 Constant series

A uniform framework often yields constant descriptors. The exact distribution is a delta PMF. Normalized autocorrelation is undefined for zero-variance series and is reported explicitly by TS3.

11.7 Correlated frames

Population standard deviations describe the analyzed trajectory. They are not standard errors and do not assume independent samples.

11.8 Large edge unions

Edge occupancy may be large for highly reactive catalogs. Users may disable it with `include_edge_occupancies=False`.

12 Validation requirements

The TS2 test suite must cover:

- uniform and partitioned topology catalogs;
- trajectory and ensemble semantics;
- exact graph-descriptor PMFs;
- component and isolated-vertex changes;
- endpoint-species counts;
- complete reverse bridge equivalence;
- asymmetric linker-order separation;
- parallel projected edges;
- degree statistics by species and persistent atom;
- canonical edge occupancy;
- trajectory aggregate additions and removals;
- disabled optional outputs;
- per-frame catalog mode;
- serialization, digest tampering, and read-only arrays;
- real Na-LTA acceptance behavior.

13 Na-LTA acceptance case

For the 2,000-frame 300 K Na-LTA framework catalog, TS2 must report:

framework topology classes:	1
vertices per frame:	48
projected edges per frame:	96
components per frame:	1
isolated vertices:	0
cycle-space rank:	49
Al-Si endpoint edges:	96
Al-O-Si bridge signatures:	96
Al degree distribution:	delta at 4
Si degree distribution:	delta at 4
canonical framework edge occupancy:	1.0 for all 96 edges
framework transition aggregate:	absent

This validates that Na contact motion does not contaminate framework statistics.

14 Deferred features

The following remain outside TS2:

- exact transition timelines and matrices;
- dwell and recurrence intervals;
- projected-edge lifetime episodes;
- autocorrelation and survival functions;
- cross-layer atomic-to-framework transition classification;
- plotting and tabular export helpers;
- primitive-ring, ring-incidence, site, and cage statistics;
- approximate topology clustering or graph-distance metrics.

15 Implementation checklist

Before modifying TS2, verify:

- Is the input a completed `TopologyCatalog`?
- Are framework vertices aligned across classes?
- Is the metric class-level, frame-level, or trajectory-only?
- Does the statistic preserve the complete bridge path modulo whole reversal?
- Is `FrameworkEdgeKey` retained for edge occupancy?
- Is cycle rank labeled correctly?
- Is ensemble order being kept non-temporal?
- Are optional large outputs controlled by options?
- Are source schemas and digests recorded?
- Are all arrays immutable and serialization round-trippable?

16 Summary

TS2 is a catalog-derived framework statistics layer:

```
TopologyCatalog
|
v
class-level graph descriptors
endpoint and whole-path bridge counts
degree and edge-occupancy statistics
optional aggregate transition effects
|
v
FrameworkTopologyStatistics
```

It provides compact, reproducible validation of projected framework behavior while preserving the architecture boundary required for later temporal and primitive-ring analysis.